

Guía del Estudiante — Taller: Mejora y Despliegue en InfinityFree y Relación con Arquitectura Cloud

1. Propósito del taller

Este taller los guiará paso a paso en la mejora del proyecto que subieron en la clase anterior en InfinityFree. El objetivo es que ustedes apliquen **buenas prácticas de desarrollo, seguridad y despliegue**, documenten el trabajo realizado y comprendan cómo cada mejora se relaciona con servicios y patrones utilizados en la nube (AWS, Azure, GCP).

Un aspecto fundamental será la **implementación de un sistema de login en su sitio web**. Esto no es solo un ejercicio técnico, sino una práctica esencial en la construcción de aplicaciones profesionales, porque:

- Permite **controlar el acceso** a la información y proteger secciones privadas del sistema (ejemplo: ver registros solo para administradores).
- Establece una **gestión de usuarios y roles** (administrador, usuario normal), similar a cómo lo hacen servicios de identidad en la nube como **IAM (Identity and Access Management)**.
- Refuerza la **seguridad del sistema**, ya que los datos sensibles no estarán disponibles para cualquiera que entre a la URL.
- Representa un paso hacia arquitecturas escalables y seguras, que son la base de los servicios modernos en la nube.

2. Resultado de aprendizaje

Al finalizar el taller, el estudiante será capaz de:

- Mejorar una aplicación web básica en hosting gratuito, aplicando validaciones y seguridad básica.
- Organizar assets y simular su separación en servicios de almacenamiento en la nube.
- Realizar backup/exports y documentar el proceso de despliegue.
- Relacionar las mejoras con servicios de arquitectura cloud y proponer un plan de migración.

3. Indicadores de logro

1. Sitio mejorado y publicado en InfinityFree con formulario funcional y control de acceso.
2. Implementación de validaciones y uso de prepared statements.
3. Organización de assets en carpeta `static/` y justificación de equivalencia con S3/Blob/Cloud Storage.
4. Entrega de evidencias: URL pública, phpMyAdmin con ≥ 3 registros, dump.sql, codigo.zip y README.md.
5. Presentación corta y plan de migración a la nube en 4 pasos.

4. Roles y responsabilidades (por equipo)

Cada equipo debe asignar los siguientes roles y registrar los nombres en el README.md:

- **Líder / Coordinador:** coordina tareas, verifica checklist y sube entrega final.
- **Desarrollador Backend:** valida inputs, implementa prepared statements, gestiona lógica de servidor.
- **Desarrollador Frontend / UI:** mejora formularios, estilos y organiza assets estáticos.
- **Administrador de BD (DBA):** exporta dump.sql, verifica integridad y crea registros de prueba.
- **DevOps / Deployment:** sube archivos a InfinityFree, configura db_connect.php y estructura public_html.
- **QA / Tester:** realiza pruebas, documenta errores y verifica correcciones.
- **Documentador / Presentador:** redacta README.md, compila evidencias y prepara presentación.

Si el equipo tiene 6 integrantes, combinar Documentador+Presentador o DBA+DevOps según habilidades.

5. Requisitos previos y herramientas

- Acceso a la cuenta de InfinityFree (crear una si no la tienen).

- Código base del proyecto subido en la clase previa.
- Editor de texto (VSCode, Sublime, Notepad++).
- Navegador con acceso a phpMyAdmin.
- Opcional: Git para control de versiones y un cliente FTP (FileZilla) si prefieren.

6. Entregables (obligatorios)

1. URL pública del sitio en InfinityFree (archivo: enlace.txt).
2. Captura phpMyAdmin con al menos 3 registros (capturas/phpmyadmin.png).
3. Captura del File Manager mostrando archivos subidos (capturas/filemanager.png).
4. dump.sql exportado desde phpMyAdmin.
5. codigo.zip con todo el proyecto.
6. README.md completo con roles, changelog y respuestas a la reflexión cloud.
7. qa-report.md con las pruebas realizadas.

7. Desarrollo paso a paso (detallado)

Sigue estas instrucciones en el orden indicado. Cada paso incluye qué archivo tocar y qué evidencias generar.

7.1 Preparación (10 minutos)

1. Revisen en el equipo el sitio que subieron en la clase pasada y abran el README.md preliminar.
2. Confirмен roles y escriban los nombres en el README.md.
3. Asegúrense de tener acceso al panel de InfinityFree y a phpMyAdmin.

7.2 Organizar assets estáticos (15–25 minutos)

Objetivo: mover CSS, imágenes y scripts a una carpeta `static/` para simular separación entre app y almacenamiento de objetos en la nube.

Pasos:

- En su proyecto, crear carpeta `static/` dentro de `public\_html` o `htdocs`.
- Mover `style.css`, logos y todas las imágenes a `static/`.

- Actualizar rutas en `index.php` y otros archivos (ej: `

Evidencia: screenshot del File Manager mostrando la carpeta `static/`.

Documentar en README: "assets movidos a static/ (simula S3/Blob/Cloud Storage)".

7.3 Validaciones y seguridad básica (25–35 minutos)

Objetivo: evitar que datos inválidos o maliciosos ingresen en su BD y proteger contraseñas.

Pasos:

- En `procesar.php`, validar los campos: `trim()`, `strlen()` para longitud máxima y `filter\_var(\$email, FILTER\_VALIDATE\_EMAIL)` para email.

- Usar `prepared statements` para todas las consultas a la BD (INSERT, SELECT).

- Si implementan login: almacenar contraseñas con `password\_hash()` y verificar con `password\_verify()` al iniciar sesión.

Código de ejemplo (inserción segura):

```
```php
$stmt = $mysqli->prepare("INSERT INTO contactos (nombre, correo, mensaje)
VALUES (?, ?, ?)");
$stmt->bind_param("sss", $nombre, $correo, $mensaje);
$stmt->execute();
$stmt->close();
```
```

Evidencia: versión mejorada de `procesar.php`, explicación en README y pruebas en qa-report.md.

7.4 Autenticación básica (opcional para equipos intermedios/avanzados) (20–30 minutos)

Objetivo: proteger la vista de registros (listar.php) con un login sencillo y roles (admin/usuario).

Pasos:

- Crear tabla `users` con campos: id, username, password_hash, role.

- Crear `register.php` (opcional) y `login.php`.
- Proteger `listar.php` usando `session\_start()` y verificando `\$\_SESSION["role"] == "admin"`.

Seguridad recomendada: llamar a `session\_regenerate\_id(true)` al iniciar sesión y usar HTTPS si está disponible.

Evidencia: screenshot mostrando login funcionando y usuario admin.

7.5 Export / Backup (15 minutos)

Objetivo: simular backups y restauraciones.

Pasos:

- En phpMyAdmin, exportar la base de datos como `dump.sql` y descargarlo.
- Implementar un botón “Exportar CSV” en `listar.php` que genere y descargue un CSV con los registros.

Código ejemplo para exportar CSV:

```
```php
header("Content-Type: text/csv; charset=utf-8");
header("Content-Disposition: attachment; filename=export_contactos.csv");
// output rows
```
```

Evidencia: archivo `dump.sql` y CSV generado.

7.6 Subida y verificación en InfinityFree (20–30 minutos)

Objetivo: desplegar cambios y comprobar que todo funciona en el hosting público.

Pasos:

1. Entrar al panel de InfinityFree y abrir File Manager.
2. Subir los archivos actualizados (o comprimir y subir zip y luego descomprimir).
3. Editar `db\_connect.php` con los datos reales (host, user, pass, db).
4. Probar el sitio en la URL pública y ejecutar los formularios.
5. Verificar en phpMyAdmin que los registros nuevos aparecen (al menos 3).

Evidencia: URL en `enlace.txt`, screenshot del File Manager y phpMyAdmin.

8. QA / Pruebas (durante todo el desarrollo)

El rol QA debe ejecutar todas las pruebas del `qa-report.md` y documentar:

- Validaciones de formulario (email vacío, email inválido).
- Intentos de inyección SQL (ej: `` OR 1=1 --``).
- Prueba de login con credenciales correctas e incorrectas.
- Export CSV y descarga correcta.

Evidencia: `qa-report.md` completado con resultados y screenshots de fallos y correcciones.

9. Documentación final y README (20 minutos)

El Documentador debe completar el `README.md` con:

- Integrantes y roles.
- Descripción del proyecto y sector elegido.
- Instrucciones para ejecutar localmente y valores a cambiar en `db\_connect.php` (sin credenciales reales).
- Evidencias (enlace y capturas).
- Changelog (qué hizo cada integrante).
- Respuestas a las preguntas de reflexión (despliegue, limitaciones, equivalentes cloud, plan de migración).

10. Checklist de entrega (líder verifica antes de enviar)

- [] URL pública (enlace.txt) funcionando.
- [] dump.sql incluido.
- [] capturas: phpMyAdmin, File Manager, sitio funcionando.
- [] codigo.zip con todo el proyecto.
- [] README.md completado.
- [] qa-report.md completado y adjuntado.

- [] Presentación breve preparada (3–4 minutos).

11. Cómo relacionar esto con Arquitectura Cloud (breve explicación para el estudiante)

1 Separación de assets (static/)

- **Lo que hicieron:** crearon una carpeta static/ para guardar imágenes, estilos y archivos separados del código principal.
- **En la nube:** eso mismo lo hacen servicios como **S3 (AWS)**, **Blob Storage (Azure)** o **Cloud Storage (GCP)**.
- **¿Por qué es importante?**
 - Los archivos se guardan en servidores especializados.
 - Se replican en varias ubicaciones (si se cae uno, sigue disponible en otro).
 - Pueden entregarse más rápido a usuarios de diferentes países gracias a redes de distribución de contenido (CDN).

👉 En palabras simples: ustedes hicieron una “mini versión” de algo que en la nube escala a nivel mundial.

2 Validaciones y prepared statements

- **Lo que hicieron:** validaron los datos que ingresan los usuarios y usaron consultas seguras para que nadie pueda hackear su base de datos con un “truco de SQL”.
- **En la nube:** además de esas validaciones, existen servicios como **WAF (Web Application Firewall)** y reglas de seguridad (**Security Groups en AWS, NSG en Azure**).
- **¿Por qué es importante?**
 - Protege la aplicación de ataques comunes (SQL injection, XSS, etc.).
 - Da control de qué tráfico puede entrar o salir de los servidores.

👉 En pocas palabras: validaciones + seguridad en la nube = aplicaciones más seguras.

3 Login y roles

- **Lo que hicieron:** crearon un login básico con usuarios y roles (ej: admin/usuario).
- **En la nube:** esto lo maneja **IAM (Identity and Access Management)** o gestores de secretos como **Key Vault** o **Secrets Manager**.
- **¿Por qué es importante?**
 - Controla exactamente quién puede hacer qué dentro del sistema.
 - Permite que un usuario tenga permisos limitados y un administrador tenga control total.
 - En empresas grandes, esto asegura que no todos los empleados accedan a todo.

👉 Es la diferencia entre darle la llave de toda la casa a cualquiera... o dar solo la llave del cuarto que necesita.

4 Export / dump.sql

- **Lo que hicieron:** exportaron la base de datos manualmente como un archivo .sql para tener un respaldo.
- **En la nube:** servicios como **RDS (AWS)** o **Cloud SQL (GCP)** hacen **backups automáticos** y pueden recuperar datos si algo falla.
- **¿Por qué es importante?**
 - Evita la pérdida de información.
 - Permite restaurar la base de datos en caso de accidente o ataque.

👉 Es como tener un seguro de tu información: si se daña, puedes volver al último punto guardado.

5 Subida manual de archivos

- **Lo que hicieron:** entraron al File Manager de InfinityFree y subieron los archivos uno por uno.
- **En la nube:** esto se reemplaza por **pipelines de CI/CD** o servicios que automatizan el despliegue como **Elastic Beanstalk (AWS)**, **App Service (Azure)** o **App Engine (GCP)**.

- **¿Por qué es importante?**

- Los cambios se suben automáticamente cada vez que un programador actualiza el código.
- Se evita el error humano de olvidar un archivo o romper la app al subir mal los archivos.

👉 Imaginen que en vez de cargar un archivo a la vez, tuvieran una “máquina automática” que despliega su proyecto en segundos sin intervención.

6 No control de red en hosting compartido

- **Lo que hicieron:** en InfinityFree no tienen control de la red; cualquiera que tenga la URL puede intentar acceder.
- **En la nube:** existen **VPC (Virtual Private Cloud en AWS y GCP)** o **VNet (Azure)** con reglas de seguridad.
- **¿Por qué es importante?**
 - Separa qué partes de la aplicación son públicas (ej: página web) y qué partes son privadas (ej: base de datos).
 - Solo ciertos servicios pueden hablar entre sí.
 - Se reducen riesgos de ataques.

👉 Es como si tuvieras tu casa dentro de una urbanización con guardas, en vez de tenerla sola y sin rejas en la mitad de la nada.

12. Recomendaciones y buenas prácticas

- No incluyan contraseñas reales en README ni en archivos subidos.
- Siempre hagan backup antes de cambios importantes.
- Validar todos los inputs en cliente y servidor.
- Comentar el código y dejar un changelog claro.
- Si pueden, prueben la migración en cuentas free-tier de AWS/Azure/GCP en futuras sesiones.

13. Recursos y referencias rápidas

- InfinityFree: <https://infinityfree.net>

- PHP Prepared Statements: <https://www.php.net/manual/en/mysqli.prepare.php>
- password_hash / password_verify:
<https://www.php.net/manual/en/function.password-hash.php>
- AWS S3 / RDS / EC2 docs (buscar en aws.amazon.com)
- Azure Blob / Azure SQL / Azure VMs (buscar en docs.microsoft.com)
- Google Cloud Storage / Cloud SQL / Compute Engine (buscar en cloud.google.com)

14. Formato de entrega

Suban un ZIP con la siguiente estructura:

miProyecto/

├─ codigo/

├─ dump.sql

├─ README.md

├─ qa-report.md

├─ capturas/ (phpmyadmin.png, filemanager.png, sitio.png)

└─ enlace.txt



Entrega del Taller

Todos los equipos deben **subir los entregables completos** (código, dump.sql, README.md, qa-report.md, capturas y demás evidencias) en la siguiente carpeta de Google Drive:



Enlace de entrega: [Carpeta Drive del Taller](#)



Recuerden nombrar su carpeta con el **nombre del equipo o del proyecto** para identificar fácilmente cada entrega.



Importante: Verifiquen que todos los archivos solicitados estén incluidos antes de subirlos. Solo se evaluarán los trabajos que estén correctamente organizados y entregados en la fecha indicada.

Presentación Disponible: *Hosting Tradicional vs Arquitectura Cloud (AWS, Azure, GCP)*

Les comparto la guía visual con ejemplos que usaremos como apoyo en nuestras clases:

[Ver la presentación “Hosting Tradicional vs Arquitectura Cloud \(AWS, Azure, GCP\) – Guía desde Cero con Ejemplos”](#)

¿Qué encontrarán allí?

- Comparación clara entre **hosting tradicional** y arquitecturas modernas en la nube.
 - Ejemplos concretos de cómo los conceptos que aplicaron en InfinityFree se transforman en servicios profesionales.
 - Almacenamiento de archivos → S3 / Blob / Cloud Storage
 - Hostings → EC2 / VM / Compute Engine
 - Bases de datos → RDS / Azure SQL / Cloud SQL
 - Redes y seguridad → VPC / NSG / VPC Network
 - Visualizaciones que les ayudarán a **entender mejor el puente entre su proyecto básico y una arquitectura escalable en la nube.**
-

Recomendaciones para usarla

- Révisenla antes de la clase para tener contexto.
- Durante la exposición, pueden ir señalando ejemplos que ya vieron en su proyecto.
- Tomen capturas o anoten conceptos clave para su README del taller.

Nos servirá como guía para entender *por qué* la nube hace lo que hace, y cómo lo que ustedes hicieron en InfinityFree es solo una versión muy básica.

¡Buen trabajo! Cualquier duda la comentan con su docente durante la sesión virtual.